

BENCHALAL Nael
LOUAILECHE AXEL

Atelier Pipeline as Code

Nous n'avons pas pu effectuer les requêtes impliquant les deux tables en correspondance, car les données récupérées ne couvrent pas la même période. L'API que nous utilisons collecte des données en temps réel, ce qui signifie qu'il n'est pas possible de récupérer des données antérieures. En revanche, les données des "Yellow Taxis" (issues du fichier TLC Trip Record Data) sont figées dans un fichier Parquet, limité à un seul mois d'historique.

De ce fait, les requêtes nécessitant la jonction ou la comparaison des deux sources de données ne sont pas réalisables. Nous avons préparé un document Word afin d'y intégrer des captures d'écran illustrant ces limitations.

Distribution des durées de trajets

```
SELECT
    trip_duration_minutes,
    COUNT(*) AS nb_trajets
FROM fact_taxi_trips
GROUP BY trip_duration_minutes
ORDER BY trip_duration_minutes;
```

	trip_duration_minutes	nb
1	-51472,31666666667	1
2	-56,03333333333333	1
3	-54,5	1
4	-45,01666666666667	1
5	-29,75	1
6	-17,2	1
7	-0,9333333333333333	1
8	-0,9	1
9	-0,85	1
10	-0,8333333333333333	2
11	-0,8166666666666667	2
12	-0,8	2
13	-0,7833333333333333	4
14	-0,75	1
15	-0,7166666666666667	3
16	-0,7	1

Longs trajets = plus de pourboires ?

```
SELECT
    CASE
        WHEN trip_distance <= 2 THEN '0-2 km'
        WHEN trip_distance > 2 AND trip_distance <= 5 THEN '2-5 km'
        ELSE '>5 km'
    END AS distance_range,
    AVG(tip_percentage) AS pourboire_moyen
FROM fact_taxi_trips
GROUP BY
    CASE
        WHEN trip_distance <= 2 THEN '0-2 km'
        WHEN trip_distance > 2 AND trip_distance <= 5 THEN '2-5 km'
        ELSE '>5 km'
    END;
```

Non pas forcément, ceux de 0-2km en obtiennent plus

	distance_range	pourboire_moyen
1	>5 km	14,0865370245383
2	2-5 km	15,86879020106
3	0-2 km	21,9224607601802

Heures de prise en charge les plus chargées

```
[ ] SELECT
    DATEPART(HOUR, tpep_pickup_datetime) AS pickup_hour,
    COUNT(*) AS nb_trajets
FROM fact_taxi_trips
GROUP BY DATEPART(HOUR, tpep_pickup_datetime)
ORDER BY nb_trajets DESC;
```

Il s'agit souvent des fins de d'après-midi (18h,19h,20h)

	pickup_hour	nb_trajets
1	19	267951
2	18	253518
3	20	221055
4	17	217051
5	16	213694
6	22	205978
7	15	202289
8	21	195001
9	14	186144
10	23	182214
11	13	175432
12	12	160076
13	11	148316
14	10	142877
15	9	141305
16	0	136812

Corrélation distance / pourcentage de pourboire

```
from pyspark.ml.stat import Correlation
from pyspark.ml.feature import VectorAssembler

assembler = VectorAssembler(inputCols=["trip_distance", "tip_percentage"], outputCol="features")
corr_df = assembler.transform(taxi_df).select("features")
Correlation.corr(corr_df, "features").show()
```

```
+-----+
| pearson(features)|
+-----+
|1.0              ...|
+-----+
```

	trip_distance	avg_tip
1	0	18,9998297324591
2	0,01	3,26749527309423
3	0,02	5,14270272035026
4	0,03	6,4225261199547
5	0,04	7,29994096467597
6	0,05	8,64441431181172
7	0,06	8,89783877516576
8	0,07	7,42612291486107
9	0,08	10,3407876168615
10	0,09	9,08857367451708
11	0,1	18,4889778292849
12	0,11	12,073062860948
13	0,12	22,4454980559665
14	0,13	14,8850510754403
15	0,14	15,9680865801888
16	0,15	15,0670194134372
17	0,16	16,0000000000000

2.1. Température moyenne lors des pics de trajets

```

WITH trajets_par_heure AS (
    SELECT
        DATEPART(HOUR, tpep_pickup_datetime) AS pickup_hour,
        COUNT(*) AS nb_trajets
    FROM fact_taxi_trips
    GROUP BY DATEPART(HOUR, tpep_pickup_datetime)
),
top_heures AS (
    SELECT TOP 5 pickup_hour FROM trajets_par_heure ORDER BY nb_trajets DESC
)
SELECT
    w.pickup_hour,
    AVG(w.temperature) AS temperature_moyenne
FROM (
    SELECT
        DATEPART(HOUR, timestamp) AS pickup_hour,
        temperature
    FROM dim_weather
) w
JOIN top_heures h ON w.pickup_hour = h.pickup_hour
GROUP BY w.pickup_hour;

```

00 %

Résultats Messages

pickup_hour	temperature_moyenne
-------------	---------------------

2.2. Impact du vent ou de la pluie sur le nombre de trajets

```
SELECT
    weather_category,
    AVG(vent_vitesse) AS vent_moyen,
    COUNT(tpep_pickup_datetime) AS nb_trajets
FROM dim_weather w
JOIN fact_taxi_trips t
    ON CAST(t.tpep_pickup_datetime AS DATE) = CAST(w.timestamp AS DATE)
    AND DATEPART(HOUR, t.tpep_pickup_datetime) = DATEPART(HOUR, w.timestamp)
GROUP BY weather_category;
```

%

Résultats

Messages

weather_category

vent_moyen

nb_trajets

3.1. Comportements selon les types de météo

```
SELECT
    weather_category,
    COUNT(*) AS nb_trajets,
    AVG(trip_distance) AS distance_moyenne,
    AVG(trip_duration_minutes) AS duree_moyenne,
    AVG(tip_percentage) AS pourboire_moyen
FROM dim_weather w
JOIN fact_taxi_trips t
    ON CAST(t.tpep_pickup_datetime AS DATE) = CAST(w.timestamp AS DATE)
    AND DATEPART(HOUR, t.tpep_pickup_datetime) = DATEPART(HOUR, w.timestamp)
GROUP BY weather_category;
```

▼

←

é

sultats

Messages

weather_category

nb_trajets

distance_moyenne

duree_moyenne

pourboire_moyen

3.2. Heure avec le plus de clients à haute valeur

```
WITH high_value_customers AS (  
    SELECT  
        passenger_count AS client_id,  
        COUNT(*) AS nb_trajets,  
        SUM(total_amount) AS depense,  
        AVG(tip_percentage) AS avg_tip  
    FROM fact_taxi_trips  
    GROUP BY passenger_count  
    HAVING COUNT(*) > 10  
        AND SUM(total_amount) > 300  
        AND AVG(tip_percentage) > 15  
)  
SELECT  
    DATEPART(HOUR, tpep_pickup_datetime) AS pickup_hour,  
    COUNT(*) AS nb_trajets_haute_valeur  
FROM fact_taxi_trips  
WHERE passenger_count IN (SELECT client_id FROM high_value_customers)  
GROUP BY DATEPART(HOUR, tpep_pickup_datetime)  
ORDER BY nb_trajets_haute_valeur DESC;
```

%	
Résultats Messages	
pickup_hour	nb_trajets_haute_valeur
18	215350
19	213969
17	198891
16	196329
15	186445
20	181472
14	173190
22	167735
21	164154
13	163777
12	150220
23	139463

Nous voyons que c'est pendant la fin d'après-midi (18h,19h,17h)

3.3. Influence de la météo sur le comportement des pourboires

```
SELECT
    weather_category,
    AVG(tip_percentage) AS pourboire_moyen
FROM dim_weather w
JOIN fact_taxi_trips t
    ON CAST(t.tpep_pickup_datetime AS DATE) = CAST(w.timestamp AS DATE)
    AND DATEPART(HOUR, t.tpep_pickup_datetime) = DATEPART(HOUR, w.timestamp)
GROUP BY weather_category
ORDER BY pourboire_moyen DESC;
```

%	
Résultats	Messages
weather_category	pourboire_moyen